# Tutorial Playwright — Sprint 1 — Conjunto 1

# Empresa: CH Hospitalar Comércio de Equipamentos Médicos e Hospitalares Ltda.

**Data:** 01/04/2026 **Dados:** dadosempportpar-1.md **Referência:** CASOS DE USO EMPRESA PORTFOLIO PARAMETRIZACAO V2.md **UCs:** F01–F17 (17 casos de uso) **Público:** Engenheiro QA / script Playwright automatizado

---

## Credenciais e Fluxo de Acesso

| Campo | Valor |
|---|---|
| Usuário (Conjunto 1) | valida1@valida.com.br |
| Senha | 123456 |
| Perfil | Superusuário |
| Empresa | CH Hospitalar Comércio de Equipamentos Médicos e Hospitalares Ltda. |

### Fluxo de login automatizado (helpers.ts)

O helper `login(page)` em `tests/e2e/playwright/helpers.ts`: 1. Limpa localStorage 2. Preenche `valida1@valida.com.br` / `123456` 3. Detecta tela de seleção de empresa 4. Seleciona CH Hospitalar (por texto ou via API switch-empresa com ID fixo) 5. Aguarda Dashboard carregar

### Menus extras visíveis (superusuário)

- **Usuarios** — CRUD de usuários do sistema
- **Associar Empresa/Usuario** — vincular usuários a empresas
- **Selecionar Empresa** — trocar empresa ativa

> Esses menus não aparecem para usuários normais (super=False).

---

## Pré-requisito CRÍTICO — Vinculação usuário↔empresa após criação

**Cenário "user novo sem vínculo" (FA-07 do UC-F01):**

Quando o usuário (super) é criado sem nenhum vínculo em `usuario_empresa`, ao logar ele cai na tela **"Você não tem empresas vinculadas"** com 3 botões: Criar Nova Empresa, Vincular Empresa a Usuário, Entrar no Sistema.

Após executar **FA-07.A — Criar Nova Empresa via CRUD** (botão azul → CRUD genérico → preencher → salvar), a empresa fica **órfã**: existe em `empresas`, mas NÃO existe registro em `usuario_empresa` ligando o usuário criador a ela. Resultado: ao tentar navegar para qualquer rota protegida (`/app/empresa`, `/app/dashboard`, etc.), o `RequireEmpresa` do React redireciona de volta para "Sem empresa vinculada".

**Por isso o tutorial DEVE incluir a vinculação imediatamente após criar empresa**, executando **FA-07.B — Vincular Empresa a Usuário** (botão roxo → página `AssociarEmpresaUsuario`):

```
// Após salvar empresa via CRUD (FA-07.A passos 1-4):

// 1. Voltar para tela "Sem empresa vinculada" OU navegar via sidebar para "Associar Empresa/U
await page.click('button[data-action="associar-empresa"]');
// OU se já estiver no shell:
await page.click('.nav-item-label:has-text("Associar Empresa/Usuario")');

// 2. Esperar página AssociarEmpresaUsuario carregar
await expect(page.locator('h1:has-text("Associar Empresa / Usuário")')).toBeVisible();

// 3. Selecionar a empresa recém-criada no select "Empresa"
//    Cada option tem formato "{razao_social} — {cnpj}"
await page.locator('label:has-text("Empresa") + select').selectOption({ label: /DEMO|CH Hospit

// 4. Selecionar o usuário corrente no select "Usuário"
//    Cada option tem formato "{name} ({email})"
await page.locator('label:has-text("Usuário") + select').selectOption({ label: /valida1/ });

// 5. Selecionar papel (default: Operador)
//    await page.locator('label:has-text("Papel") + select').selectOption({ value: 'operador'

// 6. Clicar "Vincular"
await page.click('button.action-button-primary:has-text("Vincular")');

// 7. Aguardar mensagem de sucesso (verde)
await expect(page.locator('text=/Vínculo criado/')).toBeVisible();
```

**Endpoint chamado:** `POST /api/admin/associar-empresa` com `{user_id, empresa_id, papel: "operador", acao: "vincular"}` → cria registro em `editais.usuario_empresa(user_id, empresa_id, papel, ativo=True)`.

**Pós-condição:** próximo login do mesmo usuário retorna a empresa em `vinculadas` no endpoint `/api/auth/minhas-empresas`. Usuário entra direto em rotas protegidas sem cair em "Sem empresa vinculada".

> **Observação:** este passo corresponde ao **UC-F18 — Vincular empresa a usuário** (extraído do FA-07.B do UC-F01 V6 e elevado a caso de uso autônomo em V7). Relação

> UML: `UC-F01 <<uses>> UC-F18`. Tutoriais subsequentes (UC-F02, F03, F04, F05) assumem que esta vinculação já foi feita.

### Pré-requisito CRÍTICO — Selecionar empresa ativa para a sessão

**Vincular ≠ Selecionar.** O passo anterior criou o registro em `usuario_empresa` (a empresa pode ser usada). Mas o frontend ainda **não sabe qual empresa está ativa** na sessão. Sem isso: - Top-bar não mostra a empresa - Backend não tem `empresa_id` no contexto JWT - Algumas chamadas (`/api/auth/me/contexto`) retornam dados vazios - Frontend pode redirecionar para "Sem empresa vinculada" mesmo após vincular (cache desatualizado)

Após vincular, navegue até **Configurações → Selecionar Empresa** e clique no card da empresa criada:

```
// 1. Expandir secao Configuracoes na sidebar (se colapsada)
await page.click('button.nav-section-header:has-text("Configuracoes")');

// 2. Clicar item "Selecionar Empresa"
await page.click('.nav-item-label:has-text("Selecionar Empresa")');

// 3. Aguardar pagina carregar (cards das empresas vinculadas)
await expect(page.locator('h1:has-text("Selecionar Empresa")')).toBeVisible();

// 4. Clicar no card da empresa recem-vinculada
await page.click('button:has(strong:has-text("CH Hospitalar"))');
// OU pra DEMO em ciclos automatizados:
// await page.click('button:has(strong:has-text("DEMO"))');

// 5. Aguardar empresa ficar ATIVA (badge no card)
await expect(page.locator('button:has-text("ATIVA")')).toBeVisible();
```

**Endpoint chamado:** `selecionarEmpresa(id)` do `AuthContext` chama `POST /api/auth/switch-empresa` ou similar, atualiza JWT com `empresa_id`, salva em localStorage e re-renderiza shell autenticado com a empresa ativa.

**Pós-condição:** top-bar mostra nome da empresa, backend usa `empresa_id` em todas as chamadas seguintes, navegação para `/app/empresa`, `/app/dashboard` funciona sem cair em "Sem empresa vinculada".

---

## Índice


1. UC-F01 — Cadastro Principal da Empresa
2. UC-F02 — Contatos, Emails e Área Padrão
3. UC-F03 — Upload de Documentos da Empresa
4. UC-F04 — Certidões Automáticas e Manuais
5. UC-F05 — Cadastro de Responsáveis
6. UC-F06 — Listar e Filtrar Produtos do Portfolio
7. UC-F07 — Cadastro de Produto por IA

8. UC-F08 — Editar Produto Existente
9. UC-F09 — Reprocessar Metadados por IA
10. UC-F10 — ANVISA e Busca Web de Produto
11. UC-F11 — Verificar Completude do Produto
12. UC-F12 — Metadados e Captação do Produto
13. UC-F13 — Classificação Hierárquica de Produtos
14. UC-F14 — Configurar Pesos e Limiares do Score
15. UC-F15 — Parâmetros Comerciais
16. UC-F16 — Fontes de Busca e Palavras-chave
17. UC-F17 — Notificações e Preferências do Sistema
18. Dependências entre UCs
19. Ordem de Execução Recomendada
20. Fixtures Necessários

---

## [UC-F01] Cadastro Principal da Empresa

**Página:** `EmpresaPage` — rota `/app/empresa` **Pré-condições:** - Usuário autenticado no sistema (valida1@valida.com.br, empresa: CH Hospitalar) - Empresa sem razão social cadastrada ou com campos em branco - Sidebar expandida na seção "Configuracoes"

---

## Sequência de Automação

```
// Navegar até a página Empresa via sidebar
await page.click('button:has-text("Configuracoes")');
await page.click('button:has-text("Empresa")');
await expect(page).toHaveURL(/\/app\/empresa/);

// Preencher Dados Cadastrais principais
await page.getByLabel('Razão Social').fill('CH Hospitalar Comércio de Equipamentos Médicos e H
await page.getByLabel('Nome Fantasia').fill('CH Hospitalar');
await page.getByLabel('CNPJ').fill('43.712.232/0001-85');
await page.getByLabel('Inscrição Estadual').fill('123.456.789.000');
await page.getByLabel('Website').fill('https://www.chhospitalar.com.br');

// Preencher redes sociais
await page.getByLabel('Instagram').fill('@chhospitalar');
await page.getByLabel('LinkedIn').fill('ch-hospitalar-equipamentos');
await page.getByLabel('Facebook').fill('CH HospitalarHospitalar');

// Preencher endereço
await page.getByLabel('Endereço').fill('Av. das Indústrias, 2500, Bloco B, Sala 301');
await page.getByLabel('Cidade').fill('São Paulo');
await page.getByLabel('UF').fill('SP');
await page.getByLabel('CEP').fill('04766-000');

// Salvar
await page.click('button:has-text("Salvar Alterações")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Expandir seção Configuracoes na sidebar | `button:has-text("Configuracoes")` | — | Submenus visíveis |
| 2 | Clicar em Empresa | `button:has-text("Empresa")` | — | URL contém `/app/empresa` |
| 3 | Preencher Razão Social | `page.getByLabel('Razão Social')` | `CH Hospitalar Comércio de Equipamentos Médicos e Hospitalares Ltda.` | Campo com valor correto |
| 4 | Preencher Nome Fantasia | `page.getByLabel('Nome Fantasia')` | `CH Hospitalar` | Campo com valor correto |
| 5 | Preencher CNPJ | `page.getByLabel('CNPJ')` | `43.712.232/0001-85` | Máscara aplicada corretamente |
| 6 | Preencher Inscrição Estadual | `page.getByLabel('Inscrição Estadual')` | `123.456.789.000` | Campo preenchido |
| 7 | Preencher Website | `page.getByLabel('Website')` | `https://www.chhospitalar.com.br` | Campo preenchido |
| 8 | Preencher Instagram | `page.getByLabel('Instagram')` | `@chhospitalar` | Campo preenchido |
| 9 | Preencher LinkedIn | `page.getByLabel('LinkedIn')` | `ch-hospitalar-equipamentos` | Campo preenchido |
| 10 | Preencher Facebook | `page.getByLabel('Facebook')` | `CH HospitalarHospitalar` | Campo preenchido |
| 11 | Preencher Endereço | `page.getByLabel('Endereço')` | `Av. das Indústrias, 2500, Bloco B, Sala 301` | Campo preenchido |
| 12 | Preencher Cidade | `page.getByLabel('Cidade')` | `São Paulo` | Campo preenchido |
| 13 | Preencher UF | `page.getByLabel('UF')` | `SP` | Valor SP selecionado |
| 14 | Preencher CEP | `page.getByLabel('CEP')` | `04766-000` | Máscara aplicada |
| 15 | Clicar Salvar Alterações | `button:has-text("Salvar Alterações")` | — | Toast de sucesso visível |

### Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.getByDisplayValue('CH Hospitalar Comércio de Equipamentos Médicos e Hospital
await expect(page.getByDisplayValue('43.712.232/0001-85')).toBeVisible();
await expect(page.getByDisplayValue('04766-000')).toBeVisible();
```

> **Nota para execução manual:** Conferir que todos os campos estão preenchidos e que o toast de confirmação apareceu na parte superior da tela após clicar em Salvar.

---

## [UC-F02] Contatos, Emails e Área Padrão

**Página:** `EmpresaPage` — rota `/app/empresa` **Pré-condições:** - UC-F01 concluído com sucesso - Página EmpresaPage aberta, seção de contatos localizada

---

### Sequência de Automação

```
// Localizar e rolar até a seção de Contatos/Emails
await page.locator('text=Contatos').scrollIntoViewIfNeeded();

// Adicionar primeiro email
await page.getByLabel('Email').first().fill('licitacoes@chhospitalar.com.br');

// Adicionar segundo email — clicar no botão + ou "Adicionar Email"
await page.click('button:has-text("Adicionar Email")');
await page.getByLabel('Email').nth(1).fill('comercial@chhospitalar.com.br');

// Adicionar terceiro email
await page.click('button:has-text("Adicionar Email")');
await page.getByLabel('Email').nth(2).fill('fiscal@chhospitalar.com.br');

// Adicionar primeiro telefone
await page.getByLabel('Telefone').first().fill('(11) 3456-7890');

// Adicionar segundo telefone
await page.click('button:has-text("Adicionar Telefone")');
await page.getByLabel('Telefone').nth(1).fill('(11) 98765-4321');

// Selecionar Área Padrão
await page.selectOption('select[name*="area"]', 'Equipamentos Médico-Hospitalares');
// alternativa se for campo de texto com autocomplete:
// await page.getByLabel('Área Padrão').fill('Equipamentos Médico-Hospitalares');

// Salvar
await page.click('button:has-text("Salvar Alterações")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Rolar até seção Contatos | `text=Contatos` | — | Seção de emails/ telefones visível |
| 2 | Preencher email 1 | `page.getByLabel('Email').first()` | `licitacoes@chhospitalar.com.br` | Campo preenchi |
| 3 | Clicar Adicionar Email | `button:has-text("Adicionar Email")` | — | Novo campo d email aparece |
| 4 | Preencher email 2 | `page.getByLabel('Email').nth(1)` | `comercial@chhospitalar.com.br` | Campo preenchi |
| 5 | Clicar Adicionar Email | `button:has-text("Adicionar Email")` | — | Terceiro campo d email aparece |
| 6 | Preencher email 3 | `page.getByLabel('Email').nth(2)` | `fiscal@chhospitalar.com.br` | Campo preenchi |
| 7 | Preencher telefone 1 | `page.getByLabel('Telefone').first()` | `(11) 3456-7890` | Campo preenchi |
| 8 | Clicar Adicionar Telefone | `button:has-text("Adicionar Telefone")` | — | Novo campo d telefone aparece |
| 9 | Preencher telefone 2 | `page.getByLabel('Telefone').nth(1)` | `(11) 98765-4321` | Campo preenchi |
| 10 | Selecionar Área Padrão | `select[name*="area"]` ou `getByLabel('Área Padrão')` | `Equipamentos Médico-Hospitalares` | Área selecion |
| 11 | Clicar Salvar Alterações | `button:has-text("Salvar Alterações")` | — | Toast de sucesso visível |

### Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.getByText('licitacoes@chhospitalar.com.br')).toBeVisible();
await expect(page.getByText('comercial@chhospitalar.com.br')).toBeVisible();
await expect(page.getByText('fiscal@chhospitalar.com.br')).toBeVisible();
await expect(page.getByText('(11) 3456-7890')).toBeVisible();
await expect(page.getByText('(11) 98765-4321')).toBeVisible();
await expect(page.getByText('Equipamentos Médico-Hospitalares')).toBeVisible();
```

> **Nota para execução manual:** Verificar que os três emails e dois telefones aparecem listados, e que a Área Padrão exibe "Equipamentos Médico-Hospitalares".

---

## [UC-F03] Upload de Documentos da Empresa

**Página:** `EmpresaPage` — rota `/app/empresa` (aba/seção "Documentos") **Pré-condições:** - UC-F01 concluído - Arquivo `tests/fixtures/test_document.pdf` presente no sistema de arquivos local - Data do sistema: 01/04/2026 (Doc 3 com validade 2025-12-31 estará vencido)

---

## Sequência de Automação

```
// Navegar até a seção de Documentos
await page.click('text=Documentos');
await expect(page.locator('text=Documentos da Empresa')).toBeVisible();

// --- Documento 1: Certidão / Comprovante de CNPJ ---
await page.click('button:has-text("Adicionar Documento")');
await page.selectOption('select[name*="tipo"]', 'Certidão / Comprovante de CNPJ');
await page.setInputFiles('input[type="file"]', 'tests/fixtures/test_document.pdf');
await page.fill('input[type="date"]', '2026-12-31');
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);

// --- Documento 2: Contrato Social / Estatuto (sem validade) ---
await page.click('button:has-text("Adicionar Documento")');
await page.selectOption('select[name*="tipo"]', 'Contrato Social / Estatuto');
await page.setInputFiles('input[type="file"]', 'tests/fixtures/test_document.pdf');
// deixar campo de validade em branco
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);

// --- Documento 3: Alvará de Funcionamento (validade vencida) ---
await page.click('button:has-text("Adicionar Documento")');
await page.selectOption('select[name*="tipo"]', 'Alvará de Funcionamento');
await page.setInputFiles('input[type="file"]', 'tests/fixtures/test_document.pdf');
await page.fill('input[type="date"]', '2025-12-31');
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Clicar na aba Documentos | `text=Documentos` | — | Seção de documentos visível |
| 2 | Clicar Adicionar Documento | `button:has-text("Adicionar Documento")` | — | Modal ou formulário de upload abre |
| 3 | Selecionar tipo Doc 1 | `select[name*="tipo"]` | `Certidão / Comprovante de CNPJ` | Tipo selecionado |
| 4 | Fazer upload Doc 1 | `input[type="file"]` | `tests/fixtures/test_document.pdf` | Nome do arquivo exibido |
| 5 | Preencher validade Doc 1 | `input[type="date"]` | `2026-12-31` | Data preenchida |
| 6 | Salvar Doc 1 | `button:has-text("Salvar")` | — | Toast sucesso; Doc 1 aparece na lista com badge verde (OK) |
| 7 | Clicar Adicionar Documento | `button:has-text("Adicionar Documento")` | — | Modal abre novamente |
| 8 | Selecionar tipo Doc 2 | `select[name*="tipo"]` | `Contrato Social / Estatuto` | Tipo selecionado |
| 9 | Fazer upload Doc 2 | `input[type="file"]` | `tests/fixtures/test_document.pdf` | Nome do arquivo exibido |
| 10 | Deixar validade em branco | `input[type="date"]` | (em branco) | Campo vazio |
| 11 | Salvar Doc 2 | `button:has-text("Salvar")` | — | Toast sucesso; Doc 2 na lista com badge verde (OK) |
| 12 | Clicar Adicionar Documento | `button:has-text("Adicionar Documento")` | — | Modal abre |
| 13 | Selecionar tipo Doc 3 | `select[name*="tipo"]` | `Alvará de Funcionamento` | Tipo selecionado |
| 14 | Fazer upload Doc 3 | `input[type="file"]` | `tests/fixtures/test_document.pdf` | Nome do arquivo exibido |
| 15 | Preencher validade Doc 3 | `input[type="date"]` | `2025-12-31` | Data preenchida |
| 16 | Salvar Doc 3 | `button:has-text("Salvar")` | — | Toast sucesso; Doc 3 na lista com badge amarelo (Vence) |

### Verificações finais (assertions)

```
// Verificar que os três documentos aparecem na lista
await expect(page.getByText('Certidão / Comprovante de CNPJ')).toBeVisible();
await expect(page.getByText('Contrato Social / Estatuto')).toBeVisible();
await expect(page.getByText('Alvará de Funcionamento')).toBeVisible();

// Verificar status dos badges — Doc 1 e Doc 2 verdes, Doc 3 amarelo
const rows = page.locator('table tbody tr, [data-testid="doc-row"]');
await expect(rows.nth(0).locator('.badge, [class*="status"]')).toContainText(/OK|verde/i);
await expect(rows.nth(1).locator('.badge, [class*="status"]')).toContainText(/OK|verde/i);
await expect(rows.nth(2).locator('.badge, [class*="status"]')).toContainText(/Vence|amarelo|wa
```

> **Nota para execução manual:** Conferir que a lista exibe três linhas de documentos. Doc 1 e Doc 2 com badge verde "OK". Doc 3 com badge amarelo "Vence" (validade 2025-12-31 já expirada em relação à data atual 01/04/2026).

---

## [UC-F04] Certidões Automáticas e Manuais

**Página:** `CertidoesPage` — rota `/app/certidoes` (ou aba "Certidões" dentro de EmpresaPage) **Pré-condições:** - UC-F01 concluído (CNPJ cadastrado: `43.712.232/0001-85`) - Conexão com internet disponível para busca automática (se offline, usar fallback manual)

> **Aviso:** A busca automática de certidões depende de conexão com a internet e, em alguns casos, resolução de CAPTCHA via CapSolver. Em ambientes offline ou CI/CD sem acesso externo, pule o Passo 3 e use diretamente o upload manual (Passos 6–10).

---

## Sequência de Automação

```
// Navegar até a página de Certidões
await page.click('text=Certidões');
await expect(page).toHaveURL(/\/app\/certidoes/);

// Configurar CNPJ e frequência de busca automática
await page.getByLabel('CNPJ').fill('43.712.232/0001-85');
await page.selectOption('select[name*="frequencia"]', 'Semanal');

// Disparar busca automática (pode levar 10–30s dependendo da disponibilidade das APIs)
await page.click('button:has-text("Buscar Certidões")');
await page.waitForResponse(
  (resp) => resp.url().includes('/certidoes') && resp.status() === 200,
  { timeout: 30000 }
);
// ou aguardar elemento de resposta:
await expect(page.locator('[data-testid="certidoes-list"], table')).toBeVisible({ timeout: 300

// Upload manual de certidão (fallback ou complemento)
await page.click('button:has-text("Adicionar Certidão")');
await page.setInputFiles('input[type="file"]', 'tests/fixtures/test_document.pdf');
await page.fill('input[type="date"]', '2026-06-30');
await page.getByLabel('Número').fill('CND-2025-12345');
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);

// Editar certidão via modal de detalhe
await page.click('button:has-text("Editar"), [data-testid="btn-editar-certidao"]');
await page.selectOption('select[name*="status"]', 'Valida');
await page.fill('input[type="date"]', '2026-06-30');
await page.getByLabel('Número').fill('CND-2025-12345');
await page.getByLabel('Órgão Emissor').fill('Receita Federal do Brasil');
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Certidões | `text=Certidões` | — | URL contém `/app/certidoes` |
| 2 | Preencher CNPJ | `page.getByLabel('CNPJ')` | `43.712.232/0001-85` | Campo preenchido |
| 3 | Selecionar Frequência | `select[name*="frequencia"]` | `Semanal` | Opção selecionada |
| 4 | Clicar Buscar Certidões | `button:has-text("Buscar Certidões")` | — | Loader visível; aguardar resposta (até 30s) |
| 5 | Verificar lista de certidões | `table, [data-testid="certidoes-list"]` | — | Pelo menos uma certidão listada |
| 6 | Clicar Adicionar Certidão (manual) | `button:has-text("Adicionar Certidão")` | — | Modal ou formulário abre |
| 7 | Fazer upload do arquivo | `input[type="file"]` | `tests/fixtures/test_document.pdf` | Nome do arquivo exibido |
| 8 | Preencher Data de Validade | `input[type="date"]` | `2026-06-30` | Data preenchida |
| 9 | Preencher Número da Certidão | `page.getByLabel('Número')` | `CND-2025-12345` | Campo preenchido |
| 10 | Salvar certidão manual | `button:has-text("Salvar")` | — | Toast sucesso; certidão na lista |
| 11 | Abrir modal de edição | `button:has-text("Editar")` | — | Modal de detalhe abre |
| 12 | Definir Status | `select[name*="status"]` | `Valida` | Status selecionado |
| 13 | Confirmar Validade | `input[type="date"]` | `2026-06-30` | Data confirmada |
| 14 | Confirmar Número | `page.getByLabel('Número')` | `CND-2025-12345` | Campo com valor correto |
| 15 | Preencher Órgão Emissor | `page.getByLabel('Órgão Emissor')` | `Receita Federal do Brasil` | Campo preenchido |
| 16 | Salvar edição | `button:has-text("Salvar")` | — | |

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| | | | | Toast sucesso; dados atualizados na lista |

### Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.getByText('CND-2025-12345')).toBeVisible();
await expect(page.getByText('Receita Federal do Brasil')).toBeVisible();
await expect(page.getByText(/Valida|Válida/i)).toBeVisible();
```

> **Nota para execução manual:** Verificar que a lista de certidões contém pelo menos a certidão manual com número CND-2025-12345 e status "Valida". Se a busca automática retornar certidões adicionais, conferir que estão listadas com status correto.

---

## [UC-F05] Cadastro de Responsáveis

**Página:** `EmpresaPage` — rota `/app/empresa` (seção/aba "Responsáveis") **Pré-condições:** - UC-F01 concluído - Seção de Responsáveis acessível dentro de EmpresaPage

---

### Sequência de Automação

```
// Navegar até a seção de Responsáveis
await page.click('text=Responsáveis');
await expect(page.locator('text=Responsáveis da Empresa')).toBeVisible();

// --- Responsável 1: Representante Legal ---
await page.click('button:has-text("Adicionar Responsável")');
await page.selectOption('select[name*="tipo"]', 'Representante Legal');
await page.getByLabel('Nome').fill('Marcos Antonio Ferreira');
await page.getByLabel('Cargo').fill('Diretor Executivo');
await page.getByLabel('Email').fill('diego.munoz@chhospitalar.com.br');
await page.getByLabel('Telefone').fill('(11) 98765-4321');
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);

// --- Responsável 2: Preposto ---
await page.click('button:has-text("Adicionar Responsável")');
await page.selectOption('select[name*="tipo"]', 'Preposto');
await page.getByLabel('Nome').fill('Carla Regina Souza');
await page.getByLabel('Cargo').fill('Gerente de Licitações');
await page.getByLabel('Email').fill('carla.souza@chhospitalar.com.br');
await page.getByLabel('Telefone').fill('(11) 3456-7891');
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);

// --- Responsável 3: Responsável Técnico ---
await page.click('button:has-text("Adicionar Responsável")');
await page.selectOption('select[name*="tipo"]', 'Responsável Técnico');
await page.getByLabel('Nome').fill('Dr. Paulo Roberto Menezes');
await page.getByLabel('Cargo').fill('Engenheiro Biomédico');
await page.getByLabel('Email').fill('paulo.menezes@chhospitalar.com.br');
await page.getByLabel('Telefone').fill('(11) 3456-7892');
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Clicar na aba Responsáveis | `text=Responsáveis` | — | Seção visível |
| 2 | Clicar Adicionar Responsável | `button:has-text("Adicionar Responsável")` | — | Modal abre |
| 3 | Selecionar Tipo Resp 1 | `select[name*="tipo"]` | `Representante Legal` | Tipo selecionado |
| 4 | Preencher Nome Resp 1 | `page.getByLabel('Nome')` | `Marcos Antonio Ferreira` | Campo preenchido |
| 5 | Preencher Cargo Resp 1 | `page.getByLabel('Cargo')` | `Diretor Executivo` | Campo preenchido |
| 6 | Preencher Email Resp 1 | `page.getByLabel('Email')` | `diego.munoz@chhospitalar.com.br` | Campo preenchido |
| 7 | Preencher Tel Resp 1 | `page.getByLabel('Telefone')` | `(11) 98765-4321` | Campo preenchido |
| 8 | Salvar Resp 1 | `button:has-text("Salvar")` | — | Toast sucesso; Resp 1 na lista |
| 9 | Clicar Adicionar Responsável | `button:has-text("Adicionar Responsável")` | — | Modal abre |
| 10 | Selecionar Tipo Resp 2 | `select[name*="tipo"]` | `Preposto` | Tipo selecionado |
| 11 | Preencher Nome Resp 2 | `page.getByLabel('Nome')` | `Carla Regina Souza` | Campo preenchido |
| 12 | Preencher Cargo Resp 2 | `page.getByLabel('Cargo')` | `Gerente de Licitações` | Campo preenchido |
| 13 | Preencher Email Resp 2 | `page.getByLabel('Email')` | `carla.souza@chhospitalar.com.br` | Campo preenchido |
| 14 | Preencher Tel Resp 2 | `page.getByLabel('Telefone')` | `(11) 3456-7891` | Campo preenchido |
| 15 | Salvar Resp 2 | `button:has-text("Salvar")` | — | Toast sucesso; Resp 2 na lista |
| 16 | Clicar Adicionar Responsável | `button:has-text("Adicionar Responsável")` | — | Modal abre |
| 17 | | `select[name*="tipo"]` | `Responsável Técnico` | |

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| | Selecionar Tipo Resp 3 | | | Tipo selecionado |
| 18 | Preencher Nome Resp 3 | `page.getByLabel('Nome')` | `Dr. Paulo Roberto Menezes` | Campo preenchido |
| 19 | Preencher Cargo Resp 3 | `page.getByLabel('Cargo')` | `Engenheiro Biomédico` | Campo preenchido |
| 20 | Preencher Email Resp 3 | `page.getByLabel('Email')` | `paulo.menezes@chhospitalar.com.br` | Campo preenchido |
| 21 | Preencher Tel Resp 3 | `page.getByLabel('Telefone')` | `(11) 3456-7892` | Campo preenchido |
| 22 | Salvar Resp 3 | `button:has-text("Salvar")` | — | Toast sucesso; Resp 3 na lista |

### Verificações finais (assertions)

```
await expect(page.getByText('Marcos Antonio Ferreira')).toBeVisible();
await expect(page.getByText('Representante Legal')).toBeVisible();
await expect(page.getByText('Carla Regina Souza')).toBeVisible();
await expect(page.getByText('Preposto')).toBeVisible();
await expect(page.getByText('Dr. Paulo Roberto Menezes')).toBeVisible();
await expect(page.getByText('Responsável Técnico')).toBeVisible();
```

> **Nota para execução manual:** Verificar que os três responsáveis aparecem na lista com nome, cargo e tipo corretos.

---

## [UC-F06] Listar e Filtrar Produtos do Portfolio

**Página:** `PortfolioPage` — rota `/app/portfolio` **Pré-condições:** - UC-F07 concluído (produto "Monitor Multiparamétrico BedStar-700" cadastrado) - PortfolioPage acessível via sidebar

---

### Sequência de Automação

```
// Navegar até Portfolio
await page.click('text=Portfolio');
await expect(page).toHaveURL(/\/app\/portfolio/);
await expect(page.locator('table, [data-testid="portfolio-table"]')).toBeVisible();

// Filtrar por Área
await page.selectOption('select[name*="area"], [data-testid="filtro-area"]',
  'Equipamentos Médico-Hospitalares');

// Filtrar por Classe
await page.selectOption('select[name*="classe"], [data-testid="filtro-classe"]',
  'Equipamentos de Diagnóstico por Imagem');

// Filtrar por Subclasse
await page.selectOption('select[name*="subclasse"], [data-testid="filtro-subclasse"]',
  'Ultrassonógrafo');

// Verificar que a lista filtrada exibe produtos relevantes
await expect(page.locator('table tbody tr').first()).toBeVisible();

// Busca por texto livre
await page.getByPlaceholder(/buscar|pesquisar|search/i).fill('ultrassom');
await expect(page.locator('table tbody tr')).not.toHaveCount(0);

// Limpar filtro e buscar "monitor"
await page.getByPlaceholder(/buscar|pesquisar|search/i).clear();
await page.getByPlaceholder(/buscar|pesquisar|search/i).fill('monitor');
await expect(page.locator('table tbody tr')).not.toHaveCount(0);
```

### Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Portfolio | `text=Portfolio` | — | URL contém `/app/portfolio`; tabela visível |
| 2 | Filtrar por Área | `select[name*="area"]` | `Equipamentos Médico-Hospitalares` | Lista atualizada |
| 3 | Filtrar por Classe | `select[name*="classe"]` | `Equipamentos de Diagnóstico por Imagem` | Lista filtrada |
| 4 | Filtrar por Subclasse | `select[name*="subclasse"]` | `Ultrassonógrafo` | Lista filtrada |
| 5 | Verificar resultados filtrados | `table tbody tr` | — | Pelo menos 1 produto na lista |
| 6 | Digitar busca por texto | `getByPlaceholder(/buscar/)` | `ultrassom` | Resultados com "ultrassom" visíveis |
| 7 | Limpar e buscar "monitor" | `getByPlaceholder(/buscar/)` | `monitor` | Resultados com "monitor" visíveis |

### Verificações finais (assertions)

```
await expect(page.locator('table tbody tr')).not.toHaveCount(0);
// Verificar que o produto cadastrado no UC-F07 está presente
await expect(page.getByText('Monitor Multiparamétrico BedStar-700')).toBeVisible();
```

> **Nota para execução manual:** Confirmar que os filtros reduzem a lista corretamente e que a busca por texto "ultrassom" retorna produtos relevantes.

---

## [UC-F07] Cadastro de Produto por IA

**Página:** `PortfolioPage` — rota `/app/portfolio` (modal ou subpágina de cadastro) **Pré-condições:** - Usuário autenticado - Conexão com internet disponível para processamento IA

> **Aviso:** O processamento por IA pode levar 30 segundos ou mais dependendo do modelo e da carga do servidor. Use `await page.waitForTimeout(30000)` ou `await`

> `page.waitForResponse(...)` conforme abaixo. Em ambientes sem internet, a Opção A (Website) falhará — use a Opção B (Manual Técnico com arquivo local).

---

### Sequência de Automação

```
// Navegar até Portfolio e abrir modal de cadastro
await page.click('text=Portfolio');
await expect(page).toHaveURL(/\/app\/portfolio/);
await page.click('button:has-text("Adicionar Produto"), button:has-text("Novo Produto")');

// === OPÇÃO A — Website do Fabricante ===
await page.selectOption('select[name*="tipo_documento"]', 'Website');
await page.getByLabel('URL do Website').fill(
  'https://www.mindray.com/en/products/ultrasound/general-imaging.html'
);
await page.selectOption('select[name*="area"]', 'Equipamentos Médico-Hospitalares');
await page.selectOption('select[name*="classe"]', 'Equipamentos de Diagnóstico por Imagem');
await page.selectOption('select[name*="subclasse"]', 'Ultrassonógrafo');
await page.click('button:has-text("Processar"), button:has-text("Analisar com IA")');

// Aguardar resposta da IA (pode demorar até 30s)
await page.waitForResponse(
  (resp) => resp.url().includes('/ai') || resp.url().includes('/produto'),
  { timeout: 60000 }
);
await expect(page.locator('[data-testid="produto-nome"], input[name*="nome"]'))
  .not.toBeEmpty({ timeout: 30000 });

await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);

// === OPÇÃO B — Manual Técnico (arquivo PDF) ===
await page.click('button:has-text("Adicionar Produto"), button:has-text("Novo Produto")');
await page.selectOption('select[name*="tipo_documento"]', 'Manual Técnico');
await page.setInputFiles('input[type="file"]', 'tests/fixtures/test_document.pdf');
await page.getByLabel('Nome do Produto').fill('Monitor Multiparamétrico BedStar-700');
await page.selectOption('select[name*="area"]', 'Equipamentos Médico-Hospitalares');
await page.selectOption('select[name*="classe"]', 'Monitoração');
await page.selectOption('select[name*="subclasse"]', 'Monitor Multiparamétrico');
await page.click('button:has-text("Processar"), button:has-text("Analisar com IA")');

// Aguardar resposta da IA
await page.waitForResponse(
  (resp) => resp.url().includes('/ai') || resp.url().includes('/produto'),
  { timeout: 60000 }
);
// ou aguardar elemento
await expect(page.locator('[data-testid="produto-criado"], .produto-card'))
  .toBeVisible({ timeout: 60000 });

await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Portfolio | `text=Portfolio` | — | URL contém `/app/portfolio` |
| 2 | Clicar Adicionar Produto | `button:has-text("Adicionar Produto")` | — | Modal/formulário de cadastro abre |
| 3 | Selecionar tipo Website (Opção A) | `select[name*="tipo_documento"]` | `Website` | Tipo selecionado |
| 4 | Preencher URL | `getByLabel('URL do Website')` | `https://www.mindray.com/en/products/ultrasound/general-imaging.html` | URL preenchida |
| 5 | Selecionar Área (Opção A) | `select[name*="area"]` | `Equipamentos Médico-Hospitalares` | Área selecionada |
| 6 | Selecionar Classe (Opção A) | `select[name*="classe"]` | `Equipamentos de Diagnóstico por Imagem` | Classe selecionada |
| 7 | Selecionar Subclasse (Opção A) | `select[name*="subclasse"]` | `Ultrassonógrafo` | Subclasse selecionada |
| 8 | Clicar Processar/Analisar | `button:has-text("Processar")` | — | Loader visível; aguardar até 60s |
| 9 | Verificar campos preenchidos pela IA | `input[name*="nome"]` | — | Campos auto-preenchidos pela IA |
| 10 | Salvar produto (Opção A) | `button:has-text("Salvar")` | — | Toast sucesso; produto na lista |
| 11 | Abrir novo cadastro (Opção B) | `button:has-text("Adicionar Produto")` | — | Modal abre |
| 12 | Selecionar tipo Manual (Opção B) | `select[name*="tipo_documento"]` | `Manual Técnico` | Tipo selecionado |

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 13 | Upload arquivo (Opção B) | `input[type="file"]` | `tests/fixtures/test_document.pdf` | Nome do arquivo exibido |
| 14 | Preencher Nome Produto (Opção B) | `getByLabel('Nome do Produto')` | `Monitor Multiparamétrico BedStar-700` | Campo preenchido |
| 15 | Selecionar Área (Opção B) | `select[name*="area"]` | `Equipamentos Médico-Hospitalares` | Área selecionada |
| 16 | Selecionar Classe (Opção B) | `select[name*="classe"]` | `Monitoração` | Classe selecionada |
| 17 | Selecionar Subclasse (Opção B) | `select[name*="subclasse"]` | `Monitor Multiparamétrico` | Subclasse selecionada |
| 18 | Clicar Processar/ Analisar | `button:has-text("Processar")` | — | Loader; aguardar até 60s |
| 19 | Salvar produto (Opção B) | `button:has-text("Salvar")` | — | Toast sucesso; produto na lista |

### Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.getByText('Monitor Multiparamétrico BedStar-700')).toBeVisible();
```

> **Nota para execução manual:** Após o processamento da IA, verificar que os campos do produto foram preenchidos automaticamente antes de clicar em Salvar. O processamento pode demorar mais de 30 segundos — aguardar sem fechar o modal.

---

## [UC-F08] Editar Produto Existente

**Página:** `PortfolioPage` — rota `/app/portfolio` (modal de edição do produto) **Pré-condições:** - UC-F07 concluído — produto "Monitor Multiparamétrico BedStar-700" cadastrado - Produto visível na lista do portfolio

---

## Sequência de Automação

```
// Navegar para Portfolio e localizar o produto alvo
await page.click('text=Portfolio');
await expect(page).toHaveURL(/\/app\/portfolio/);
await page.getByPlaceholder(/buscar|pesquisar/i).fill('Monitor Multiparamétrico BedStar-700');
await expect(page.getByText('Monitor Multiparamétrico BedStar-700')).toBeVisible();

// Abrir modal de edição
await page.click('button:has-text("Editar"), [data-testid="btn-editar"]');
await expect(page.locator('[role="dialog"], .modal')).toBeVisible();

// Editar campos principais
await page.getByLabel('Nome').clear();
await page.getByLabel('Nome').fill('Monitor Multiparamétrico BedStar-700 Plus');
await page.getByLabel('Fabricante').clear();
await page.getByLabel('Fabricante').fill('BedStar Medical International');
await page.getByLabel('Modelo').clear();
await page.getByLabel('Modelo').fill('BedStar-700 Plus');
await page.getByLabel('SKU').clear();
await page.getByLabel('SKU').fill('BST700-PLUS-BR');
await page.getByLabel('NCM').clear();
await page.getByLabel('NCM').fill('9018.19.90');
await page.getByLabel('Descrição').clear();
await page.getByLabel('Descrição').fill(
  'Monitor multiparamétrico para UTI e pronto-socorro, 7 parâmetros simultâneos'
);

// Classificação
await page.selectOption('select[name*="area"]', 'Equipamentos Médico-Hospitalares');
await page.selectOption('select[name*="classe"]', 'Monitoração');
await page.selectOption('select[name*="subclasse"]', 'Monitor Multiparamétrico');

// Especificações técnicas
await page.getByLabel('Nº parâmetros').fill('7');
await page.getByLabel('Display').fill('10,4 polegadas');
await page.selectOption('select[name*="spo2"]', 'Sim');
await page.getByLabel('ECG').fill('Sim (6 derivações)');
await page.selectOption('select[name*="nibp"]', 'Sim');
await page.getByLabel('Bateria').fill('4 horas');
await page.getByLabel('Peso').fill('3,2 kg');
await page.getByLabel('Alimentação').fill('100-240 V');
await page.getByLabel('Registro ANVISA').fill('80262090001');

// Salvar edição
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Portfolio | `text=Portfolio` | — | URL contém `/app/portfolio` |
| 2 | Buscar produto alvo | `getByPlaceholder(/buscar/)` | `Monitor Multiparamétrico BedStar-700` | Produto visível na lista |
| 3 | Clicar Editar | `button:has-text("Editar")` | — | Modal de edição abre |
| 4 | Editar Nome | `getByLabel('Nome')` | `Monitor Multiparamétrico BedStar-700 Plus` | Campo atualizado |
| 5 | Editar Fabricante | `getByLabel('Fabricante')` | `BedStar Medical International` | Campo atualizado |
| 6 | Editar Modelo | `getByLabel('Modelo')` | `BedStar-700 Plus` | Campo atualizado |
| 7 | Editar SKU | `getByLabel('SKU')` | `BST700-PLUS-BR` | Campo atualizado |
| 8 | Editar NCM | `getByLabel('NCM')` | `9018.19.90` | Campo atualizado |
| 9 | Editar Descrição | `getByLabel('Descrição')` | `Monitor multiparamétrico para UTI e pronto-socorro, 7 parâmetros simultâneos` | Campo atualizado |
| 10 | Selecionar Área | `select[name*="area"]` | `Equipamentos Médico-Hospitalares` | Área selecionada |
| 11 | Selecionar Classe | `select[name*="classe"]` | `Monitoração` | Classe selecionada |
| 12 | Selecionar Subclasse | `select[name*="subclasse"]` | `Monitor Multiparamétrico` | Subclasse selecionada |
| 13 | Preencher Nº parâmetros | `getByLabel('Nº parâmetros')` | `7` | Campo preenchido |
| 14 | Preencher Display | `getByLabel('Display')` | `10,4 polegadas` | Campo preenchido |
| 15 | Preencher SpO2 | `select[name*="spo2"]` | `Sim` | Opção selecionada |
| 16 | Preencher ECG | `getByLabel('ECG')` | `Sim (6 derivações)` | Campo preenchido |
| 17 | Preencher NIBP | `select[name*="nibp"]` | `Sim` | Opção selecionada |

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 18 | Preencher Bateria | `getByLabel('Bateria')` | `4 horas` | Campo preenchido |
| 19 | Preencher Peso | `getByLabel('Peso')` | `3,2 kg` | Campo preenchido |
| 20 | Preencher Alimentação | `getByLabel('Alimentação')` | `100-240 V` | Campo preenchido |
| 21 | Preencher Registro ANVISA | `getByLabel('Registro ANVISA')` | `80262090001` | Campo preenchido |
| 22 | Salvar | `button:has-text("Salvar")` | — | Toast sucesso; produto atualizado na lista |

### Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.getByText('Monitor Multiparamétrico BedStar-700 Plus')).toBeVisible();
await expect(page.getByText('BedStar Medical International')).toBeVisible();
await expect(page.getByText('BST700-PLUS-BR')).toBeVisible();
```

> **Nota para execução manual:** Verificar que o produto na lista do portfolio passou a exibir o nome "Monitor Multiparamétrico BedStar-700 Plus" e que o fabricante é "BedStar Medical International".

---

## [UC-F09] Reprocessar Metadados por IA

**Página:** `PortfolioPage` — rota `/app/portfolio` (botão na DataTable) **Pré-condições:** - UC-F08 concluído — produto "Monitor Multiparamétrico BedStar-700 Plus" cadastrado e salvo - Conexão com internet disponível para processamento IA

> **Aviso:** O reprocessamento por IA pode levar 30 segundos ou mais. Use `waitForResponse` ou `waitForTimeout(30000)` para aguardar a conclusão antes de verificar os resultados.

---

## Sequência de Automação

```
// Navegar para Portfolio e localizar o produto
await page.click('text=Portfolio');
await expect(page).toHaveURL(/\/app\/portfolio/);
await page.getByPlaceholder(/buscar|pesquisar/i).fill('Monitor Multiparamétrico BedStar-700 Pl
await expect(page.getByText('Monitor Multiparamétrico BedStar-700 Plus')).toBeVisible();

// Clicar em Reprocessar IA na linha da DataTable
await page.locator('tr:has-text("Monitor Multiparamétrico BedStar-700 Plus")')
  .locator('button:has-text("Reprocessar IA"), [data-testid="btn-reprocessar"]')
  .click();

// Aguardar confirmação (se houver dialog de confirmação)
const confirmDialog = page.locator('[role="dialog"]:has-text("Reprocessar")');
if (await confirmDialog.isVisible()) {
  await page.click('button:has-text("Confirmar"), button:has-text("Sim")');
}

// Aguardar resposta da IA — pode demorar até 60s
await page.waitForResponse(
  (resp) =>
    (resp.url().includes('/reprocessar') || resp.url().includes('/ai')) &&
    resp.status() === 200,
  { timeout: 60000 }
);
// alternativa: aguardar elemento de feedback
await expect(
  page.locator('.toast, [role="alert"], [data-testid="reprocessamento-concluido"]')
).toBeVisible({ timeout: 60000 });
```

### Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Portfolio | `text=Portfolio` | — | URL contém `/app/portfolio` |
| 2 | Buscar produto alvo | `getByPlaceholder(/buscar/)` | `Monitor Multiparamétrico BedStar-700 Plus` | Produto visível na tabela |
| 3 | Clicar Reprocessar IA na linha | `tr:has-text("...")` `button:has-text("Reprocessar IA")` | — | Loader ou toast "Processando..." |
| 4 | Confirmar dialog (se aparecer) | `button:has-text("Confirmar")` | — | Dialog fechado |
| 5 | Aguardar resposta IA | `waitForResponse(...)` ou `waitForTimeout(30000)` | — | Toast de conclusão visível (até 60s) |
| 6 | Verificar lista atualizada | `table tbody tr:has-text("BedStar-700 Plus")` | — | Linha do produto atualizada |

### Verificações finais (assertions)

```
await expect(
  page.locator('.toast, [role="alert"]')
).toContainText(/reprocessado|sucesso|concluído/i, { timeout: 60000 });
await expect(page.getByText('Monitor Multiparamétrico BedStar-700 Plus')).toBeVisible();
```

> **Nota para execução manual:** Clicar em "Reprocessar IA" na linha do produto na DataTable. Aguardar a notificação de conclusão antes de verificar os dados atualizados. O processamento pode demorar até 1 minuto.

---

## [UC-F10] ANVISA e Busca Web de Produto

**Página:** `PortfolioPage` ou modal de detalhe do produto — rota `/app/portfolio` **Pré-condições:** - UC-F08 concluído — produto "Monitor Multiparamétrico BedStar-700 Plus" com Registro ANVISA `80262090001` - Conexão com internet disponível

---

## Sequência de Automação

```
// Navegar para Portfolio e abrir detalhe do produto
await page.click('text=Portfolio');
await page.getByPlaceholder(/buscar|pesquisar/i).fill('Monitor Multiparamétrico BedStar-700 Pl
await page.locator('tr:has-text("Monitor Multiparamétrico BedStar-700 Plus")')
  .locator('button:has-text("Detalhes"), a:has-text("Ver")')
  .click();

// Busca ANVISA pelo número de registro
await page.getByLabel('Número ANVISA').fill('80262090001');
await page.getByLabel('Nome ANVISA').fill('Monitor Multiparamétrico BedStar');
await page.click('button:has-text("Buscar ANVISA"), button:has-text("Consultar ANVISA")');
await page.waitForResponse(
  (resp) => resp.url().includes('/anvisa') && resp.status() === 200,
  { timeout: 30000 }
);
await expect(page.locator('[data-testid="anvisa-resultado"], .anvisa-card')).toBeVisible({ tim

// Busca Web por nome de produto
await page.getByLabel('Nome para Busca Web').fill('Ultrassonógrafo Portátil Mindray M7');
await page.getByLabel('Fabricante').fill('Mindray');
await page.click('button:has-text("Buscar na Web"), button:has-text("Pesquisar Web")');
await page.waitForResponse(
  (resp) => resp.url().includes('/busca-web') || resp.url().includes('/web-search'),
  { timeout: 30000 }
);
await expect(page.locator('[data-testid="web-resultado"], .web-result-card')).toBeVisible({ ti
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Portfolio | `text=Portfolio` | — | URL contém `/app/portfolio` |
| 2 | Abrir detalhe do produto | `button:has-text("Detalhes")` na linha do produto | — | Página/modal de detalhe abre |
| 3 | Preencher Número ANVISA | `getByLabel('Número ANVISA')` | `80262090001` | Campo preenchido |
| 4 | Preencher Nome ANVISA | `getByLabel('Nome ANVISA')` | `Monitor Multiparamétrico BedStar` | Campo preenchido |
| 5 | Clicar Buscar ANVISA | `button:has-text("Buscar ANVISA")` | — | Loader; aguardar até 30s |
| 6 | Verificar resultado ANVISA | `[data-testid="anvisa-resultado"]` | — | Card com dados ANVISA visível |
| 7 | Preencher Nome Busca Web | `getByLabel('Nome para Busca Web')` | `Ultrassonógrafo Portátil Mindray M7` | Campo preenchido |
| 8 | Preencher Fabricante | `getByLabel('Fabricante')` | `Mindray` | Campo preenchido |
| 9 | Clicar Buscar na Web | `button:has-text("Buscar na Web")` | — | Loader; aguardar até 30s |
| 10 | Verificar resultado Web | `[data-testid="web-resultado"]` | — | Card com resultados web visível |

## Verificações finais (assertions)

```
await expect(page.locator('[data-testid="anvisa-resultado"], .anvisa-card')).toBeVisible();
await expect(page.getByText('80262090001')).toBeVisible();
await expect(page.locator('[data-testid="web-resultado"], .web-result-card')).toBeVisible();
await expect(page.getByText(/Mindray|M7/i)).toBeVisible();
```

> **Nota para execução manual:** Verificar que o sistema retornou dados da ANVISA para o número 80262090001 e que a busca web retornou resultados para o Mindray M7.

## [UC-F11] Verificar Completude do Produto

**Página:** `PortfolioPage` ou modal de completude — rota `/app/portfolio` **Pré-condições:** - UC-F08 concluído — produto "Monitor Multiparamétrico BedStar-700 Plus" com especificações técnicas preenchidas

---

### Sequência de Automação

```
// Navegar para Portfolio e localizar produto
await page.click('text=Portfolio');
await page.getByPlaceholder(/buscar|pesquisar/i).fill('Monitor Multiparamétrico BedStar-700 Pl
await expect(page.getByText('Monitor Multiparamétrico BedStar-700 Plus')).toBeVisible();

// Abrir painel de completude
await page.locator('tr:has-text("Monitor Multiparamétrico BedStar-700 Plus")')
  .locator('button:has-text("Completude"), [data-testid="btn-completude"]')
  .click();

// Verificar scores
await expect(page.locator('[data-testid="score-geral"], .score-geral')).toBeVisible();
const scoreGeralText = await page.locator('[data-testid="score-geral"]').textContent();
const scoreGeral = parseInt(scoreGeralText || '0');
expect(scoreGeral).toBeGreaterThanOrEqual(80);

await expect(page.locator('[data-testid="score-basicos"], .score-basicos')).toBeVisible();
const scoreBasicosText = await page.locator('[data-testid="score-basicos"]').textContent();
const scoreBasicos = parseInt(scoreBasicosText || '0');
expect(scoreBasicos).toBeGreaterThanOrEqual(90);

await expect(page.locator('[data-testid="score-specs"], .score-specs')).toBeVisible();
const scoreSpecsText = await page.locator('[data-testid="score-specs"]').textContent();
const scoreSpecs = parseInt(scoreSpecsText || '0');
expect(scoreSpecs).toBeGreaterThanOrEqual(75);

// Verificar cor/badge do score geral (verde = >= 80%)
await expect(
  page.locator('[data-testid="badge-score-geral"], .badge-score')
).toHaveClass(/verde|green|success/i);
```

### Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Portfolio | `text=Portfolio` | — | URL contém `/app/portfolio` |
| 2 | Buscar produto | `getByPlaceholder(/buscar/)` | `Monitor Multiparamétrico BedStar-700 Plus` | Produto visível |
| 3 | Abrir painel de Completude | `button:has-text("Completude")` | — | Painel/modal de scores abre |
| 4 | Verificar Score Geral | `[data-testid="score-geral"]` | — | Valor >= 80% com badge verde |
| 5 | Verificar Score Básicos | `[data-testid="score-basicos"]` | — | Valor >= 90% |
| 6 | Verificar Score Specs | `[data-testid="score-specs"]` | — | Valor >= 75% |

### Verificações finais (assertions)

```
// Score Geral >= 80% (verde)
const scoreGeral = await page.locator('[data-testid="score-geral"]').textContent();
expect(parseInt(scoreGeral || '0')).toBeGreaterThanOrEqual(80);

// Score Básicos >= 90%
const scoreBasicos = await page.locator('[data-testid="score-basicos"]').textContent();
expect(parseInt(scoreBasicos || '0')).toBeGreaterThanOrEqual(90);

// Score Specs >= 75%
const scoreSpecs = await page.locator('[data-testid="score-specs"]').textContent();
expect(parseInt(scoreSpecs || '0')).toBeGreaterThanOrEqual(75);
```

> **Nota para execução manual:** O painel de completude deve mostrar Score Geral em verde (>= 80%), Score Básicos >= 90% e Score de Especificações >= 75%.

---

## [UC-F12] Metadados e Captação do Produto

**Página:** `PortfolioPage` — modal de metadados/captação do produto **Pré-condições:** - UC-F08 concluído — produto "Monitor Multiparamétrico BedStar-700 Plus" salvo

> **Aviso:** A geração de metadados por IA pode levar 20–30 segundos. Use `waitForResponse` adequado.

---

## Sequência de Automação

```
// Navegar para Portfolio e abrir metadados do produto
await page.click('text=Portfolio');
await page.getByPlaceholder(/buscar|pesquisar/i).fill('Monitor Multiparamétrico BedStar-700 Pl
await page.locator('tr:has-text("Monitor Multiparamétrico BedStar-700 Plus")')
  .locator('button:has-text("Metadados"), [data-testid="btn-metadados"]')
  .click();

// Preencher/confirmar CATMAT
await page.getByLabel('CATMAT').fill('462, 444');

// Preencher termos de captação
await page.getByLabel('Termos de Busca').fill('monitor multiparametrico');
await page.click('button:has-text("Adicionar Termo")');
await page.getByLabel('Termos de Busca').fill('monitor sinais vitais');
await page.click('button:has-text("Adicionar Termo")');
await page.getByLabel('Termos de Busca').fill('monitor uti');
await page.click('button:has-text("Adicionar Termo")');

// Aguardar processamento IA dos metadados (se aplicável)
await page.waitForResponse(
  (resp) => resp.url().includes('/metadados') && resp.status() === 200,
  { timeout: 30000 }
);

// Salvar
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Portfolio | `text=Portfolio` | — | URL contém `/app/portfolio` |
| 2 | Buscar produto | `getByPlaceholder(/buscar/)` | `Monitor Multiparamétrico BedStar-700 Plus` | Produto visível |
| 3 | Abrir modal Metadados | `button:has-text("Metadados")` | — | Modal abre |
| 4 | Preencher CATMAT | `getByLabel('CATMAT')` | `462, 444` | Campo preenchido |
| 5 | Adicionar Termo 1 | `getByLabel('Termos de Busca')` + `button:has-text("Adicionar Termo")` | `monitor multiparametrico` | Termo adicionado à lista |
| 6 | Adicionar Termo 2 | mesma lógica | `monitor sinais vitais` | Termo adicionado |
| 7 | Adicionar Termo 3 | mesma lógica | `monitor uti` | Termo adicionado |
| 8 | Aguardar IA (se aplicável) | `waitForResponse(...)` | — | Resposta recebida em até 30s |
| 9 | Salvar | `button:has-text("Salvar")` | — | Toast sucesso |

## Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.getByText('monitor multiparametrico')).toBeVisible();
await expect(page.getByText('monitor sinais vitais')).toBeVisible();
await expect(page.getByText('monitor uti')).toBeVisible();
await expect(page.getByText('462')).toBeVisible();
await expect(page.getByText('444')).toBeVisible();
```

> **Nota para execução manual:** Verificar que os três termos de busca aparecem na lista de termos de captação e que o CATMAT foi salvo com os códigos 462 e 444.

# [UC-F13] Gerir e consultar classificação Área/Classe/Subclasse

> **V8 do UC (29/04/2026):** Reescrito. Antes era apenas selecionar uma hierarquia preexistente. Agora **CRIA** Área → Classe → Subclasse via 3 CRUDs ( `empresa_scoped=True` — cada empresa tem sua própria hierarquia) e depois consulta a árvore consolidada.

**Páginas:** - `crud:areas-produto` (Cadastros → Áreas de Produto) - `crud:classes-produto-v2` (Cadastros → Classes de Produto) - `crud:subclasses-produto` (Cadastros → Subclasses de Produto) - `PortfolioPage` ( `/app/portfolio` aba Classificação)

**Pré-condições:** - UC-F01 + UC-F18 concluídos (empresa criada e vinculada ao usuário corrente) - Empresa selecionada como ativa - Sem hierarquia prévia (empresa nova) — tutorial é idempotente: pula criação se item já existe

---

## Hierarquias canônicas a criar

```
Equipamentos Médico-Hospitalares
└── Monitoração
    └── Monitor Multiparamétrico (NCM 9018.19.90)

Diagnóstico in Vitro e Laboratório
├── Reagentes Bioquímicos
│   └── Reagente para Glicose (NCM 3822.19.90)
└── Reagentes e Kits Diagnósticos
    └── Kit de Hematologia (NCM 3822.19.90)
```

Total: 2 áreas, 3 classes, 3 subclasses.

---

## Sequência de Automação Playwright

```
// ===== PARTE 1 — Áreas =====
// Sidebar: expandir Cadastros, clicar Áreas de Produto
await page.click('button.nav-section-header:has-text("Cadastros")');
await page.click('.nav-item-label:text-is("Áreas de Produto")');
await expect(page.locator('h1, h2').filter({ hasText: /Áreas de Produto/ })).toBeVisible();

// Criar Área 1: Equipamentos Médico-Hospitalares
await page.click('button:has-text("Novo")');
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Equipamentos Mé
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);

// Criar Área 2: Diagnóstico in Vitro e Laboratório
await page.click('button:has-text("Novo")');
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Diagnóstico in
await page.click('button:has-text("Salvar")');

// ===== PARTE 2 — Classes =====
await page.click('.nav-item-label:text-is("Classes de Produto")');
await expect(page.locator('h1, h2').filter({ hasText: /Classes de Produto/ })).toBeVisible();

// Classe 1: Monitoração (vinculada a Equipamentos)
await page.click('button:has-text("Novo")');
await page.selectOption('select[name="area_id"], label:has-text("Área") + select', { label: 'E
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Monitoração');
await page.click('button:has-text("Salvar")');

// Classe 2: Reagentes Bioquímicos (vinculada a Diagnóstico)
await page.click('button:has-text("Novo")');
await page.selectOption('select[name="area_id"], label:has-text("Área") + select', { label: 'D
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Reagentes Bioqu
await page.click('button:has-text("Salvar")');

// Classe 3: Reagentes e Kits Diagnósticos (vinculada a Diagnóstico)
await page.click('button:has-text("Novo")');
await page.selectOption('select[name="area_id"], label:has-text("Área") + select', { label: 'D
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Reagentes e Kit
await page.click('button:has-text("Salvar")');

// ===== PARTE 3 — Subclasses (com NCM) =====
await page.click('.nav-item-label:text-is("Subclasses de Produto")');
await expect(page.locator('h1, h2').filter({ hasText: /Subclasses de Produto/ })).toBeVisible(

// Subclasse 1: Monitor Multiparamétrico (Monitoração / NCM 9018.19.90)
await page.click('button:has-text("Novo")');
await page.selectOption('select[name="classe_id"], label:has-text("Classe") + select', { label
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Monitor Multipa
await page.locator('input[name="ncm"], label:has-text("NCM") + input').fill('9018.19.90');
await page.click('button:has-text("Salvar")');

// Subclasse 2: Reagente para Glicose (Reagentes Bioquímicos / NCM 3822.19.90)
await page.click('button:has-text("Novo")');
await page.selectOption('select[name="classe_id"], label:has-text("Classe") + select', { label
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Reagente para G
await page.locator('input[name="ncm"], label:has-text("NCM") + input').fill('3822.19.90');
await page.click('button:has-text("Salvar")');
```

```
// Subclasse 3: Kit de Hematologia (Reagentes e Kits / NCM 3822.19.90)
await page.click('button:has-text("Novo")');
await page.selectOption('select[name="classe_id"], label:has-text("Classe") + select', { label
await page.locator('input[name="nome"], label:has-text("Nome") + input').fill('Kit de Hematolo
await page.locator('input[name="ncm"], label:has-text("NCM") + input').fill('3822.19.90');
await page.click('button:has-text("Salvar")');

// ===== PARTE 4 — Visualizar árvore consolidada =====
await page.click('button.nav-section-header:has-text("Configurações")');
await page.click('.nav-item-label:text-is("Portfolio")');
await page.click('text=Classificação');
// Áreas listadas
await expect(page.getByText('Equipamentos Médico-Hospitalares')).toBeVisible();
await expect(page.getByText('Diagnóstico in Vitro e Laboratório')).toBeVisible();
// Expandir e conferir
await page.click('text=Equipamentos Médico-Hospitalares');
await expect(page.getByText('Monitoração')).toBeVisible();
await page.click('text=Monitoração');
await expect(page.getByText('Monitor Multiparamétrico')).toBeVisible();
await expect(page.getByText('9018.19.90')).toBeVisible();
```

## Passos detalhados (resumo)

| Parte | Ação | Quantidade |
|---|---|---|
| 1 — Áreas | POST `/api/crud/areas-produto` | 2 áreas |
| 2 — Classes | POST `/api/crud/classes-produto-v2` (com `area_id`) | 3 classes |
| 3 — Subclasses | POST `/api/crud/subclasses-produto` (com `classe_id` + `ncm`) | 3 subclasses |
| 4 — Visualização | GET PortfolioPage aba Classificação | 1 verificação |

## Verificações finais (asserções)

```
// Após PARTE 4
await expect(page.getByText('Equipamentos Médico-Hospitalares')).toBeVisible();
await expect(page.getByText('Monitoração')).toBeVisible();
await expect(page.getByText('Monitor Multiparamétrico')).toBeVisible();
await expect(page.getByText('Diagnóstico in Vitro e Laboratório')).toBeVisible();
await expect(page.getByText('Reagentes Bioquímicos')).toBeVisible();
await expect(page.getByText('Reagente para Glicose')).toBeVisible();
await expect(page.getByText('Reagentes e Kits Diagnósticos')).toBeVisible();
await expect(page.getByText('Kit de Hematologia')).toBeVisible();
await expect(page.getByText('9018.19.90')).toBeVisible();
await expect(page.getByText('3822.19.90')).toBeVisible();
```

> **Nota para execução manual:** UC-F13 V8 requer ~12-18 minutos. UCs subsequentes (F02, F06, F08, Sprint 2) só funcionam depois.

## [UC-F14] Configurar Pesos e Limiares do Score

**Página:** `ParametrizacaoPage` — rota `/app/parametrizacao` (seção de Pesos e Limiares) **Pré-condições:** - Usuário autenticado com permissão de configuração - Página de Parametrização acessível

### Sequência de Automação

```
// Navegar para Parametrização
await page.click('text=Parametrização');
await expect(page).toHaveURL(/\/app\/parametrizacao/);

// Localizar seção de Pesos
await page.click('text=Pesos e Limiares');

// Preencher pesos dos critérios (soma deve ser 1.00)
await page.getByLabel('Peso Técnico').clear();
await page.getByLabel('Peso Técnico').fill('0.25');
await page.getByLabel('Peso Documental').clear();
await page.getByLabel('Peso Documental').fill('0.20');
await page.getByLabel('Peso Complexidade').clear();
await page.getByLabel('Peso Complexidade').fill('0.10');
await page.getByLabel('Peso Jurídico').clear();
await page.getByLabel('Peso Jurídico').fill('0.15');
await page.getByLabel('Peso Logístico').clear();
await page.getByLabel('Peso Logístico').fill('0.15');
await page.getByLabel('Peso Comercial').clear();
await page.getByLabel('Peso Comercial').fill('0.15');

// Verificar que a soma exibida é 1.00
await expect(page.locator('[data-testid="soma-pesos"], .soma-pesos')).toContainText('1.00');

// Preencher limiares — Score Final
await page.getByLabel('Score Final GO').fill('0.70');
await page.getByLabel('Score Final NO-GO').fill('0.40');

// Limiares — Técnico
await page.getByLabel('Técnico GO').fill('0.65');
await page.getByLabel('Técnico NO-GO').fill('0.35');

// Limiares — Jurídico
await page.getByLabel('Jurídico GO').fill('0.80');
await page.getByLabel('Jurídico NO-GO').fill('0.50');

// Salvar
await page.click('button:has-text("Salvar Configurações"), button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Parametrização | `text=Parametrização` | — | URL contém `/app/parametrizacao` |
| 2 | Localizar seção Pesos | `text=Pesos e Limiares` | — | Seção de pesos visível |
| 3 | Preencher Peso Técnico | `getByLabel('Peso Técnico')` | `0.25` | Campo preenchido |
| 4 | Preencher Peso Documental | `getByLabel('Peso Documental')` | `0.20` | Campo preenchido |
| 5 | Preencher Peso Complexidade | `getByLabel('Peso Complexidade')` | `0.10` | Campo preenchido |
| 6 | Preencher Peso Jurídico | `getByLabel('Peso Jurídico')` | `0.15` | Campo preenchido |
| 7 | Preencher Peso Logístico | `getByLabel('Peso Logístico')` | `0.15` | Campo preenchido |
| 8 | Preencher Peso Comercial | `getByLabel('Peso Comercial')` | `0.15` | Campo preenchido |
| 9 | Verificar soma pesos | `[data-testid="soma-pesos"]` | — | Exibe `1.00` |
| 10 | Preencher Score Final GO | `getByLabel('Score Final GO')` | `0.70` | Campo preenchido |
| 11 | Preencher Score Final NO-GO | `getByLabel('Score Final NO-GO')` | `0.40` | Campo preenchido |
| 12 | Preencher Técnico GO | `getByLabel('Técnico GO')` | `0.65` | Campo preenchido |
| 13 | Preencher Técnico NO-GO | `getByLabel('Técnico NO-GO')` | `0.35` | Campo preenchido |
| 14 | Preencher Jurídico GO | `getByLabel('Jurídico GO')` | `0.80` | Campo preenchido |
| 15 | Preencher Jurídico NO-GO | `getByLabel('Jurídico NO-GO')` | `0.50` | Campo preenchido |
| 16 | Salvar Configurações | `button:has-text("Salvar")` | — | Toast sucesso |

## Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.getByDisplayValue('0.25')).toBeVisible(); // Peso Técnico
await expect(page.getByDisplayValue('0.70')).toBeVisible(); // Score Final GO
await expect(page.getByDisplayValue('0.80')).toBeVisible(); // Jurídico GO
// verificar soma
await expect(page.locator('[data-testid="soma-pesos"]')).toContainText('1.00');
```

> **Nota para execução manual:** Verificar que a soma dos seis pesos exibe exatamente 1.00 (ou 100%) antes de salvar. Os limiares GO e NO-GO devem ser salvos para Score Final, Técnico e Jurídico.

---

## [UC-F15] Parâmetros Comerciais

**Página:** `ParametrizacaoPage` — rota `/app/parametrizacao` (seção de Parâmetros Comerciais) **Pré-condições:** - Usuário autenticado - Página de Parametrização acessível

---

## Sequência de Automação

```
// Navegar para Parametrização
await page.click('text=Parametrização');
await expect(page).toHaveURL(/\/app\/parametrizacao/);
await page.click('text=Parâmetros Comerciais');

// Configurar Estados de atuação (NÃO marcar "Todo o Brasil")
const estados = ['SP', 'RJ', 'MG', 'RS', 'PR', 'SC', 'DF', 'GO', 'BA', 'PE'];
for (const uf of estados) {
  await page.check(`input[value="${uf}"], label:has-text("${uf}") input`);
}
// Garantir que "Todo o Brasil" NÃO está marcado
await expect(page.locator('input[value="TODOS"], label:has-text("Todo o Brasil") input'))
  .not.toBeChecked();

// Configurar Prazo e Frequência
await page.getByLabel('Prazo (dias)').fill('30');
await page.selectOption('select[name*="frequencia"]', 'Mensal');

// Configurar TAM / SAM / SOM
await page.getByLabel('TAM').fill('12500000000');
await page.getByLabel('SAM').fill('2800000000');
await page.getByLabel('SOM').fill('180000000');

// Configurar Markup, Custos Fixos e Frete
await page.getByLabel('Markup (%)').fill('28');
await page.getByLabel('Custos Fixos').fill('85000');
await page.getByLabel('Frete').fill('350');

// Ativar modalidades
const modalidades = [
  'Pregão Eletrônico',
  'Concorrência',
  'Tomada de Preços',
  'Dispensa',
];
for (const modalidade of modalidades) {
  await page.check(`input[value="${modalidade}"], label:has-text("${modalidade}") input`);
}

// Salvar
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Parametrização | `text=Parametrização` | — | URL contém `/app/parametrizacao` |
| 2 | Localizar Parâmetros Comerciais | `text=Parâmetros Comerciais` | — | Seção visível |
| 3 | Marcar estado SP | `input[value="SP"]` | — | Checkbox marcado |
| 4 | Marcar estado RJ | `input[value="RJ"]` | — | Checkbox marcado |
| 5 | Marcar estado MG | `input[value="MG"]` | — | Checkbox marcado |
| 6 | Marcar estado RS | `input[value="RS"]` | — | Checkbox marcado |
| 7 | Marcar estado PR | `input[value="PR"]` | — | Checkbox marcado |
| 8 | Marcar estado SC | `input[value="SC"]` | — | Checkbox marcado |
| 9 | Marcar estado DF | `input[value="DF"]` | — | Checkbox marcado |
| 10 | Marcar estado GO | `input[value="GO"]` | — | Checkbox marcado |
| 11 | Marcar estado BA | `input[value="BA"]` | — | Checkbox marcado |
| 12 | Marcar estado PE | `input[value="PE"]` | — | Checkbox marcado |
| 13 | Confirmar "Todo o Brasil" desmarcado | `input[value="TODOS"]` | — | Checkbox NÃO marcado |
| 14 | Preencher Prazo | `getByLabel('Prazo (dias)')` | `30` | Campo preenchido |
| 15 | Selecionar Frequência | `select[name*="frequencia"]` | `Mensal` | Opção selecionada |
| 16 | Preencher TAM | `getByLabel('TAM')` | `12500000000` | Campo preenchido |
| 17 | Preencher SAM | `getByLabel('SAM')` | `2800000000` | Campo preenchido |
| 18 | Preencher SOM | `getByLabel('SOM')` | `180000000` | Campo preenchido |
| 19 | Preencher Markup | `getByLabel('Markup (%)')` | `28` | Campo preenchido |
| 20 | Preencher Custos Fixos | `getByLabel('Custos Fixos')` | `85000` | Campo preenchido |
| 21 | Preencher Frete | `getByLabel('Frete')` | `350` | Campo preenchido |
| 22 | Ativar Pregão Eletrônico | `label:has-text("Pregão Eletrônico") input` | — | Checkbox marcado |
| 23 | Ativar Concorrência | `label:has-text("Concorrência") input` | — | Checkbox marcado |
| 24 | Ativar Tomada de Preços | `label:has-text("Tomada de Preços") input` | — | Checkbox marcado |
| 25 | Ativar Dispensa | | — | Checkbox marcado |

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| | | `label:has-text("Dispensa") input` | | |
| 26 | Salvar | `button:has-text("Salvar")` | — | Toast sucesso |

### Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
// Verificar estados selecionados
for (const uf of ['SP', 'RJ', 'MG', 'RS', 'PR', 'SC', 'DF', 'GO', 'BA', 'PE']) {
  await expect(page.locator(`input[value="${uf}"]`)).toBeChecked();
}
// Verificar "Todo o Brasil" NÃO marcado
await expect(page.locator('input[value="TODOS"]')).not.toBeChecked();
await expect(page.getByDisplayValue('30')).toBeVisible();
```

> **Nota para execução manual:** Conferir que exatamente 10 estados estão marcados (SP, RJ, MG, RS, PR, SC, DF, GO, BA, PE) e que o checkbox "Todo o Brasil" está DESMARCADO. Verificar os valores de TAM, SAM e SOM salvos.

---

## [UC-F16] Fontes de Busca e Palavras-chave

**Página:** `ParametrizacaoPage` — rota `/app/parametrizacao` (seção de Fontes e Keywords)
**Pré-condições:** - Usuário autenticado - Página de Parametrização acessível

---

### Sequência de Automação

```
// Navegar para Parametrização
await page.click('text=Parametrização');
await expect(page).toHaveURL(/\/app\/parametrizacao/);
await page.click('text=Fontes de Busca');

// Desativar BLL (toggle/switch)
const bllToggle = page.locator('label:has-text("BLL") input[type="checkbox"], [data-testid="to
await bllToggle.uncheck();
await expect(bllToggle).not.toBeChecked();

// Reativar BLL
await bllToggle.check();
await expect(bllToggle).toBeChecked();

// Preencher palavras-chave
const keywords = [
  'monitor multiparametrico',
  'ultrassonografo',
  'equipamento hospitalar',
  'material hospitalar',
  'ventilador pulmonar',
  'oximetro',
  'desfibrilador',
  'bisturi eletrico',
  'autoclave',
  'mesa cirurgica',
];
await page.getByLabel('Palavras-chave').fill(keywords.join(', '));

// Preencher NCMs
const ncms = [
  '9018.19.90',
  '9018.90.99',
  '9021.90.90',
  '9018.11.00',
  '9402.90.00',
];
await page.getByLabel('NCMs').fill(ncms.join(', '));

// Salvar
await page.click('button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Parametrização | `text=Parametrização` | — | URL contém `/app/parametrizacao` |
| 2 | Localizar Fontes de Busca | `text=Fontes de Busca` | — | Seção visível |
| 3 | Desativar BLL | `[data-testid="toggle-bll"]` ou `label:has-text("BLL") input` | — | Toggle inativo/desmarcado |
| 4 | Reativar BLL | mesmo selector | — | Toggle ativo/marcado |
| 5 | Preencher Palavras-chave | `getByLabel('Palavras-chave')` | `monitor multiparametrico, ultrassonografo, equipamento hospitalar, material hospitalar, ventilador pulmonar, oximetro, desfibrilador, bisturi eletrico, autoclave, mesa cirurgica` | Campo preenchido |
| 6 | Preencher NCMs | `getByLabel('NCMs')` | `9018.19.90, 9018.90.99, 9021.90.90, 9018.11.00, 9402.90.00` | Campo preenchido |
| 7 | Salvar | `button:has-text("Salvar")` | — | Toast sucesso |

## Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
await expect(page.locator('[data-testid="toggle-bll"]')).toBeChecked();
await expect(page.getByText('monitor multiparametrico')).toBeVisible();
await expect(page.getByText('9018.19.90')).toBeVisible();
await expect(page.getByText('9018.90.99')).toBeVisible();
```

> **Nota para execução manual:** Verificar que o BLL está ativado (toggle azul/verde) após o ciclo desativar+reativar, que as 10 palavras-chave foram salvas e que os 5 NCMs estão listados.

## [UC-F17] Notificações e Preferências do Sistema

**Página:** `ParametrizacaoPage` ou `PreferenciasPage` — rota `/app/parametrizacao` ou `/app/preferencias` **Pré-condições:** - Usuário autenticado - Email `licitacoes@chhospitalar.com.br` já cadastrado (UC-F02)

---

### Sequência de Automação

```
// Navegar até Preferências / Notificações
await page.click('text=Preferências');
// ou
await page.click('text=Notificações');
await expect(page).toHaveURL(/\/app\/(preferencias|parametrizacao|notificacoes)/);

// Configurar canais de notificação
await page.getByLabel('Email de notificação').fill('licitacoes@chhospitalar.com.br');
await page.check('input[name*="canal_email"], label:has-text("Email") input[type="checkbox"]');
await page.check('input[name*="canal_sistema"], label:has-text("Sistema") input[type="checkbox
// SMS deve estar DESMARCADO
await page.uncheck('input[name*="canal_sms"], label:has-text("SMS") input[type="checkbox"]');
await expect(
  page.locator('input[name*="canal_sms"], label:has-text("SMS") input[type="checkbox"]')
).not.toBeChecked();

// Configurar frequência de notificação
await page.selectOption('select[name*="frequencia_notif"]', 'Diario');

// Configurar tema
await page.selectOption('select[name*="tema"]', 'Escuro');

// Configurar idioma
await page.selectOption('select[name*="idioma"]', 'pt-BR');

// Configurar fuso horário
await page.selectOption('select[name*="fuso"]', 'America/Sao_Paulo');

// Salvar
await page.click('button:has-text("Salvar Preferências"), button:has-text("Salvar")');
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
```

## Passos detalhados

| # | Ação | Selector | Dado | Assert |
|---|---|---|---|---|
| 1 | Navegar para Preferências | `text=Preferências` ou `text=Notificações` | — | Página de preferênci carrega |
| 2 | Preencher email de notificação | `getByLabel('Email de notificação')` | `licitacoes@chhospitalar.com.br` | Campo preenchid |
| 3 | Ativar canal Email | `label:has-text("Email") input[type="checkbox"]` | — | Checkbox marcado (✅) |
| 4 | Ativar canal Sistema | `label:has-text("Sistema") input[type="checkbox"]` | — | Checkbox marcado (✅) |
| 5 | Desmarcar canal SMS | `label:has-text("SMS") input[type="checkbox"]` | — | Checkbox desmarcad (❌) |
| 6 | Selecionar Frequência | `select[name*="frequencia_notif"]` | `Diario` | Opção selecionad |
| 7 | Selecionar Tema | `select[name*="tema"]` | `Escuro` | Opção "Escuro" selecionad |
| 8 | Selecionar Idioma | `select[name*="idioma"]` | `pt-BR` | Opção pt-BR selecionad |
| 9 | Selecionar Fuso Horário | `select[name*="fuso"]` | `America/Sao_Paulo` | Opção selecionad |
| 10 | Salvar Preferências | `button:has-text("Salvar")` | — | Toast sucesso |

### Verificações finais (assertions)

```
await expect(page.locator('.toast, [role="alert"]')).toContainText(/salvo|sucesso/i);
// Email e Sistema marcados; SMS desmarcado
await expect(
  page.locator('label:has-text("Email") input[type="checkbox"]')
).toBeChecked();
await expect(
  page.locator('label:has-text("Sistema") input[type="checkbox"]')
).toBeChecked();
await expect(
  page.locator('label:has-text("SMS") input[type="checkbox"]')
).not.toBeChecked();
// Verificar seleções salvas
await expect(page.locator('select[name*="tema"]')).toHaveValue(/[Ee]scuro/);
await expect(page.locator('select[name*="idioma"]')).toHaveValue('pt-BR');
await expect(page.locator('select[name*="fuso"]')).toHaveValue('America/Sao_Paulo');
```

> **Nota para execução manual:** Confirmar que Email e Sistema estão com checkboxes marcados (verde/azul) e SMS desmarcado. O tema deve mudar visivelmente para "Escuro" após salvar.

---

## Dependências entre UCs

| UC | Depende de |
|---|---|
| UC-F01 | — (pré-condição: usuário autenticado) |
| UC-F02 | UC-F01 |
| UC-F03 | UC-F01 |
| UC-F04 | UC-F01 (CNPJ cadastrado) |
| UC-F05 | UC-F01 |
| UC-F06 | UC-F07 (produto na lista) |
| UC-F07 | — (independente; conexão internet para Opção A) |
| UC-F08 | UC-F07 (produto existente) |
| UC-F09 | UC-F08 (produto com nome atualizado) |
| UC-F10 | UC-F08 (Registro ANVISA `80262090001` preenchido) |
| UC-F11 | UC-F08 (especificações técnicas preenchidas) |
| UC-F12 | UC-F08 (produto salvo) |
| UC-F13 | UC-F07 ou UC-F08 (produto com classificação) |
| UC-F14 | — (configuração global; independente) |
| UC-F15 | — (configuração global; independente) |
| UC-F16 | — (configuração global; independente) |
| UC-F17 | UC-F02 (email cadastrado) |

---

## Ordem de Execução Recomendada

```
1.  UC-F01  — Cadastro Principal da Empresa
2.  UC-F02  — Contatos, Emails e Área Padrão
3.  UC-F03  — Upload de Documentos da Empresa
4.  UC-F04  — Certidões Automáticas e Manuais
5.  UC-F05  — Cadastro de Responsáveis
6.  UC-F14  — Configurar Pesos e Limiares do Score (configuração global)
7.  UC-F15  — Parâmetros Comerciais (configuração global)
8.  UC-F16  — Fontes de Busca e Palavras-chave (configuração global)
9.  UC-F17  — Notificações e Preferências do Sistema
10. UC-F07  — Cadastro de Produto por IA (Opção A + Opção B)
11. UC-F08  — Editar Produto Existente
12. UC-F13  — Classificação Hierárquica de Produtos
13. UC-F06  — Listar e Filtrar Produtos do Portfolio
14. UC-F09  — Reprocessar Metadados por IA
15. UC-F10  — ANVISA e Busca Web de Produto
16. UC-F11  — Verificar Completude do Produto
17. UC-F12  — Metadados e Captação do Produto
```

---

## Fixtures Necessários

| Fixture | Caminho | Uso |
|---|---|---|
| Documento PDF genérico | `tests/fixtures/test_document.pdf` | UC-F03, UC-F04, UC-F07 (Opção B) |
| Imagem de teste (opcional) | `tests/fixtures/test_image.png` | Substituto em caso de upload de imagem |

**Verificar existência dos fixtures antes de executar:**

```
import * as fs from 'fs';

test.beforeAll(() => {
  const fixtures = [
    'tests/fixtures/test_document.pdf',
    'tests/fixtures/test_image.png',
  ];
  for (const f of fixtures) {
    if (!fs.existsSync(f)) {
      throw new Error(`Fixture ausente: ${f}`);
    }
  }
});
```

**Credenciais de autenticação:** - Usuário: `valida1@valida.com.br` / Senha: `123456` - ID fixo da CH Hospitalar no banco: `7dbdc60a-b806-4614-a024-a1d4841dc8c9` - O helper `login()` já lida automaticamente com a tela de seleção de empresa.

**URL base:** - Backend: `http://localhost:5007` - Frontend: `http://localhost:5175` - Configurar em `playwright.config.ts`: `baseURL: 'http://localhost:5175'`